

COMPLETE PROJECT REPORT: ENTERPRISE DATA ENGINEERING PORTFOLIO

Comprehensive Architectural Design, Core Source Code Analysis, Security Hardening, and Google Cloud Platform (GCP) Multi-Service Deployment

Prepared For: Senior Engineering Panel & Public Host Distribution

Author / Engineer: Archit Somayajula

Project Focus: Full-Stack Data Pipelines, Data Quality Guardrails, Web Security Obfuscation

Cloud Providers: GCP App Engine (Standard), Firebase Hosting (Edge CDN)

Backend Engine: Flask, SQLite 3, PySpark, Docker, REST Services

Report Version: 1.1.0 (Production Final)

Document Date: June 9, 2026

1. Project Overview & Architectural Philosophy

This project showcases a complete full-stack web application designed to demonstrate industrial-grade data engineering workflows. Rather than displaying static resume details, this project hosts a series of functional, interactive tools running live in a cloud environment.

The project is designed to prove proficiency in building robust ETL pipelines, custom validation frameworks (modeled after Archit's experience at Discover Financial), containerized or serverless hosting configurations, and client-side source code obfuscation techniques. The architecture divides the codebase into a clean, stateless frontend serving user interfaces and a Python Flask API serving data queries.

System Flow Diagram

Users interact with the frontend, which requests asynchronous actions from the Flask API (such as triggering an ETL run or pulling dashboard metrics). The backend fetches live data from open APIs, runs it through the Data Quality (DQ) Engine, transforms it, and loads it into a local SQLite database. The frontend then dynamically polls these records to compile live charts and statistics, keeping the browser view completely updated.

2. Interactive Front-End UI Components

The frontend is designed to feel highly premium, using glassmorphism styling, Harmony HSL colors, Google Fonts (Plus Jakarta Sans & Space Grotesk), and SVG graphics. In mobile views, the cards automatically collapse to single columns, and code blocks scroll horizontally to fit smaller viewports (down to 320px) without breaking page layout constraints.

Interactive Sections:

- **ETL Ingestion Console (pipeline.html)**: A terminal emulator where users can trigger the pipeline and watch logs stream from the Flask backend in real time.
- **Live Analytics Dashboard (dashboard.html)**: Renders metrics from the SQLite database. Utilizes Chart.js to plot hourly air quality index (AQI) values, rolling averages, and quarantine categories.
- **Architecture Map (architecture.html)**: An interactive SVG drawing of the system flow. Clicking nodes shows specific applied skills, config files, and executes localized simulations.
- **Explainable AI Lab (xai.html)**: Details published NASA solar wind research using interactive attention connection graphs, SHAP force plot bars, and custom magnetosphere SVGs.

3. Full-Stack Data Pipeline & Ingestion Engine

The ETL pipeline core resides in backend/pipeline.py. The code executes a multi-stage ingestion job that extracts, validates, transforms, and loads air quality data.

Detailed Code Walkthrough & Source Code Reference

1. Extraction Stage (pipeline.py: L73-106):

The pipeline iterates over target locations, calling the Open-Meteo Air Quality API via HTTP GET. It extracts hourly records for PM2.5, PM10, Nitrogen Dioxide (NO2), and Ozone (O3) and returns them in a raw array of dictionaries.

2. Ingestion & Discover DQ Rules Engine (pipeline.py: L148-202):

To model real-world business constraints, we inject four simulated anomalies (negative value, null value, extreme spike outlier, and missing mandatory column) into the data stream. The engine validates each dictionary:

```
for idx, rec in enumerate(all_raw_records):
    # Rule 1: Missing Mandatory Column
    if not rec.get("city"):
        quarantine_records.append({
            "city": "UNKNOWN", "timestamp": rec.get("timestamp", "N/A"),
            "raw_payload": str(rec), "failure_reason": "Missing mandatory column 'city'"
        })
        continue
    # Rule 2: Null Value Violation
    if rec.get("pm2_5") is None or rec.get("pm10") is None:
        quarantine_records.append({
            "city": rec["city"], "timestamp": rec["timestamp"],
            "raw_payload": str(rec), "failure_reason": "Null value violation in PM2.5 or PM10"
        })
        continue
    # Rule 3: Bound Constraints
    pm2_5_val, pm10_val = rec["pm2_5"], rec["pm10"]
    if pm2_5_val < 0 or pm10_val < 0:
        quarantine_records.append({
            "city": rec["city"], "timestamp": rec["timestamp"],
            "raw_payload": str(rec), "failure_reason": f"Out-of-bounds metric (Negative PM2.5: {pm2_5_val})"
        })
        continue
    # Rule 4: Outlier Cap
    if pm2_5_val > 500 or pm10_val > 500:
        quarantine_records.append({
            "city": rec["city"], "timestamp": rec["timestamp"],
            "raw_payload": str(rec), "failure_reason": f"Outlier anomaly detected (PM2.5: {pm2_5_val} ug/m3 exceeds cap of 500)"
        })
        continue
    clean_records.append(rec)
```

3. Transformation Stage (pipeline.py: L204-228):

Passed records are converted to a Pandas DataFrame, sorted chronologically, and a rolling 3-hour average is calculated per city. The PM2.5 values are mapped to index categories (Good, Moderate, Sensitive, Unhealthy) based on US EPA standards.

4. Loading Stage (pipeline.py: L230-252):

Establishes a connection to SQLite. Clean analytical rows are appended to the processed_metrics table, and failed payloads are logged in the quarantine_records table alongside metadata.

4. Full-Stack Security Hardening & Obfuscation

Because frontend code is delivered directly to the client's browser, hiding it completely is not possible. However, we have implemented robust, full-stack defenses to protect the source code and secure the backend server.

1. Client-Side Inspect Element Prevention (security.js)

A dedicated security script (security.js) is loaded in the header of every HTML file. It blocks mouse context menus (right-clicks) and intercepts key triggers to prevent opening DevTools or viewing the page source. Additionally, it executes a background debugger loop that stalls browser execution if the inspector is opened:

```
// Disable context menu
document.addEventListener('contextmenu', event => event.preventDefault());

// Block F12, Ctrl+Shift+I (Inspect), Ctrl+Shift+J (Console), Ctrl+U (View Source)
document.addEventListener('keydown', event => {
  if (event.keyCode === 123) event.preventDefault();
  if (event.ctrlKey && event.shiftKey && (event.keyCode === 73 || event.keyCode === 74 ||
event.keyCode === 67)) event.preventDefault();
  if (event.ctrlKey && event.keyCode === 85) event.preventDefault();
});

// Active Debugger Loop Traps
(function() {
  setInterval(() => {
    try {
      Function('debugger')(); // Pauses execution if DevTools is open
    } catch (err) {}
  }, 200);
})();
```

2. Automated Obfuscation Build Pipeline (build.js)

We built a compiler script (build.js) using Node.js to bundle and protect our code. It minifies CSS and HTML, and obfuscates JavaScript using javascript-obfuscator with high-security rules:

- **Self-Defending Lock:** Modifying or formatting the code breaks its logic, crashing the browser tab.
- **Control Flow Flattening:** Scrambles the execution path, making it extremely hard to follow.
- **Console Silencing:** Automatically strips out diagnostic print logs in production.

3. Flask Backend Security Hardening (app.py)

The Flask API backend is secured against network attacks: debug mode is locked to False in production to prevent execution of arbitrary code via Flask tracebacks. CORS is locked to accept requests only from the frontend's domain, and local bindings are isolated during development.

5. Google Cloud Platform (GCP) Infrastructure

The application is hosted on Google Cloud Platform, leveraging serverless resources to run securely and stay entirely within GCP's free tier limits.

Frontend Hosting: Firebase Hosting Edge CDN

The obfuscated production assets are served via Firebase Hosting. We configure security headers inside `firebase.json` to block frame embedding, MIME-sniffing, and apply a strict Content Security Policy (CSP):

```
{
  "hosting": {
    "public": "dist",
    "headers": [
      {
        "source": "**",
        "headers": [
          { "key": "X-Frame-Options", "value": "DENY" },
          { "key": "X-Content-Type-Options", "value": "nosniff" },
          { "key": "Content-Security-Policy", "value": "default-src 'self'; script-src 'self' https://cdn.jsdelivr.net; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com https://cdnjs.cloudflare.com; connect-src 'self' https://archit-de-portfolio-8902.uc.r.appspot.com;" }
        ]
      }
    ]
  }
}
```

Backend Hosting: Google App Engine Standard

The Python Flask REST API runs on Google App Engine Standard. It uses the Python 3.12 runtime, F1 instance sizing (100% free under GCP's 28 instance-hours/day quota), and Gunicorn. GAE handles incoming request loads, queries the local SQLite database, and returns analytical responses:

```
runtime: python312
instance_class: F1
entrypoint: gunicorn -b :$PORT app:app

env_variables:
  FLASK_ENV: "production"
  ALLOWED_ORIGIN: "https://archit-de-portfolio-8902.web.app"
```

6. Tools, Libraries & Rationale Directory

The following table documents the core software tools, libraries, and frameworks utilized to construct, compile, and deploy the portfolio:

Tool / Library	Version	Rationale & Application Role
Python	3.12 / 3.13	Core backend programming language; handles ETL orchestration and API serving.
Flask	3.0.x	Micro-web framework; handles REST routing, JSON serializing, and CORS.
Pandas	2.2.x	Data analysis library; computes rolling averages and indexes index categories.
SQLite 3	Built-in	Relational database engine; stores processed metrics and quarantine tables.
Gunicorn	21.x	WSGI HTTP server; handles concurrent requests for Python on App Engine.
Node.js	v24.x	Javascript build runtime environment; executes the build.js compiler.
javascript-obfuscator	4.1.x	Node module; scrambles variables, flattens flows, and embeds debugger traps.
clean-css	5.3.x	CSS optimizer; minifies stylesheet files and cleans spacing formatting.
html-minifier-terser	7.2.x	HTML parser; strips spacing, lines, comments, and minifies inline styles.
ReportLab	4.5.x	PDF generation library; compiles this complete project documentation.

7. Custom File Ingestion & Dynamic RAG Dashboard

To support generic dataset workflows, we implemented a custom file uploader and a RAG (Retrieval-Augmented Generation) schema mapper on the Live Dashboard. This allows the portfolio to analyze, transform, and map any CSV or JSON file dynamically.

Core Functionality & Flow Details

- **Cleansing & Profiling:** Standardizes column headers, drops empty fields, and profiles data schemas using a hybrid PySpark and Pandas execution engine.
- **RAG Column Resolver:** Maps natural language queries semantically. Matches user keywords (e.g. 'dust reading') to the target column names (e.g. 'pm2_5') using synonym weights.
- **AI SQL Assistant Chat:** An agentic chat panel where users query the dataset in plain English. The resolver automatically compiles SQL queries, runs them on SQLite, and prints responses.
- **Pre-seeded Preset:** Seeding weather data automatically into the custom workspace to enable immediate querying out-of-the-box.

8. Data Engineering Glossary & Key Definitions

- **ETL (Extract, Transform, Load):** The core data integration process where data is read from sources, cleaned/aggregated, and loaded into databases.
- **Data Quality (DQ) Rules Engine:** A validation framework that inspects incoming data packets against schema constraints (missing columns, null bounds) before writing to target storages.
- **Quarantine Table:** A designated database table where records failing validation rules are isolated alongside detailed error metadata, preventing pipeline failures.
- **Code Obfuscation:** The process of transforming source code into a format that is difficult for humans to read and reverse-engineer, while keeping its functionality intact.
- **Content Security Policy (CSP):** A security HTTP response header that restricts what resources (scripts, styles, connections) can be loaded and executed by the browser.
- **CORS (Cross-Origin Resource Sharing):** A security mechanism that uses HTTP headers to determine whether a browser is permitted to read API responses initiated from a different origin domain.
- **App Engine Standard:** A serverless GCP platform that runs application instances inside sandboxed containers, scaling resources automatically based on demand.
- **Firebase Hosting:** A secure, global SSD-backed CDN hosting platform by Google, optimized for serving static frontend web pages quickly and securely.